

Lesson 6 - Pipelines

READ ME FIRST (theory). Then open the practice notebook.

Contents

Lesson 6 - Pipelines	1
Key words	1
1. The problem without a Pipeline	2
2. A Pipeline = one box	2
3. ColumnTransformer - different steps for different columns	2
4. SimpleImputer - fill missing inside the pipeline	2
5. Save and load with joblib	3
6. Does a Pipeline make the model more accurate?	3
Quick summary	3
Now do this	3

Lesson 6 - Pipelines

A Pipeline puts all the steps in ONE box. The theory, then `practice.ipynb`.

Key words

Word	Simple meaning
Pipeline	one object that holds all steps in order
ColumnTransformer	sends each group of columns to its own steps
SimpleImputer	fills missing values automatically
joblib	saves/loads a trained model to a file

1. The problem without a Pipeline

Without a Pipeline you have many loose pieces: a median value, a scaler, an encoder, a model. You must remember to use all of them, in the right order, every time. Forget one -> wrong answer, no error message.

2. A Pipeline = one box

A **Pipeline** holds all the steps (fill, scale, encode, model) in the right order, in one object.

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
```

```
numeric = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])
```

Why: one object, one file, no forgotten steps. In production you call ONE line.

3. ColumnTransformer - different steps for different columns

Numbers and text need different treatment. `ColumnTransformer` sends each group to the right mini-pipeline.

```
from sklearn.compose import ColumnTransformer
preprocessor = ColumnTransformer([
    ("num", numeric, num_cols),
    ("cat", categorical, cat_cols),
])
```

4. SimpleImputer - fill missing inside the pipeline

`SimpleImputer(strategy="median")` fills missing numbers with the median, automatically, every time. For text use `strategy="most_frequent"`.

5. Save and load with joblib

```
import joblib
joblib.dump(pipeline, "model.joblib")    # save
model = joblib.load("model.joblib")      # load later, even on another computer
```

Why: train once, save the file, use it anywhere. No need to retrain.

6. Does a Pipeline make the model more accurate?

No. Same math, same accuracy. A Pipeline makes the code **safe, short, and repeatable**. That is the point.

Quick summary

- A **Pipeline** = one box of steps in the right order.
- **ColumnTransformer** routes numbers and text to different steps.
- **SimpleImputer** fills missing values inside the pipeline.
- **joblib** saves/loads the trained pipeline.
- Pipelines do not change accuracy - they make code safe and simple.

Now do this

Open `practice.ipynb`, build a preprocessor pipeline, and do the assignment.